

Database Systems

Laboratory Work 1: Relational Model & Keys

Name: Syrym Samal

Group: Wed 15:00-16:00

Date: 16.09.2026

Part 1: Key Identification Exercises

Task 1.1: Superkey and Candidate Key Analysis

Relation A: Employee

Employee(EmplID, SSN, Email, Phone, Name, Department, Salary)

EmplID	SSN	Email	Phone	Name	Department	Salary
101	123-45-6789	john@company.com	555-0101	John	IT	75000
102	987-65-4321	mary@company.com	555-0102	Mary	HR	68000
103	456-78-9123	bob@company.com	555-0103	Bob	IT	72000

Superkeys:

1. {EmplID}
2. {SSN}
3. {Email}
4. {Phone}
5. {EmplID, Name}
6. {SSN, Department}

Candidate keys:

1. {EmplID}
2. {SSN}
3. {Email}
4. {Phone}

Which candidate key would you choose as primary key and why?

1. SSN: unique and never changes, but it is sensitive personal data — should be avoided as a PK for privacy reasons.
2. Email: unique, but people change their email addresses over time — not stable.
3. Phone: can also change, and (as discussed in Q4) may not be guaranteed unique in all cases.
4. EmplID: a surrogate key — invented specifically to identify the row, stable, contains no personal information.

Primary key: EmplID, because it is stable, does not carry sensitive data, and is guaranteed unique by design

Can two employees have the same phone number? Justify your answer based on the data shown.

In the sample data, all phone numbers happen to be different, but this does not prove Phone is a true candidate key. Two employees could share the same phone number in reality — for example, if they share a desk/office landline. Since the business rules don't guarantee phone uniqueness, Phone should not be treated as a reliable candidate key.

Relation B: Course Registration

Registration(StudentID, CourseCode, Section, Semester, Year, Grade, Credits)

Business Rules:

- A student can take the same course in different semesters
- A student cannot register for the same course section in the same semester
- Each course section in a semester has a fixed credit value

Primary Key:

Registration(StudentID, CourseCode, Section, Semester, Year, Grade, Credits)

Why each attribute is necessary:

1. StudentID — identifies which student is registering.
2. CourseCode — identifies which course; without it, Section alone is ambiguous since different courses can have a section named "A".
3. Section — needed because a student could theoretically be linked to a course through multiple sections, and sections are distinct instances of a course.
4. Semester and Year — both needed together, because "Fall" repeats every year; without Year, registrations from different years would collide. Also required because a student can retake the same course in a different semester

Additional candidate keys:

None. No other combination of attributes in this table uniquely identifies a row. Grade and Credits are dependent attributes.

Task 1.2: Foreign Key Design**Given Tables:**

Student(StudentID, Name, Email, Major, AdvisorID)

Professor(ProfID, Name, Department, Salary)

Course(CourseID, Title, Credits, DepartmentCode)

Department(DeptCode, DeptName, Budget, ChairID)

Enrollment(StudentID, CourseID, Semester, Grade)

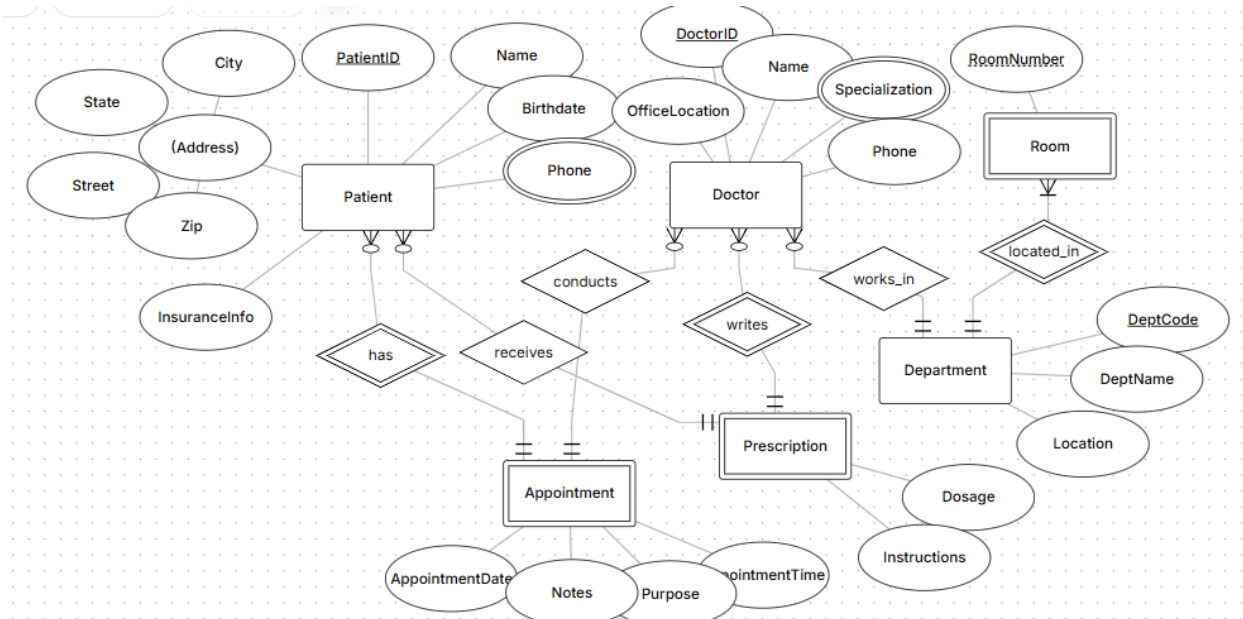
Foreign Key Relationships:

1. Student.AdvisorID -> Professor.ProfID
(each student has one advisor, who is a professor)
2. Professor.Department -> Department.DeptCode
(each professor belongs to one department)
3. Course.DepartmentCode -> Department.DeptCode
(each course is offered by one department)
4. Department.ChairID -> Professor.ProfID
(each department has one chair, who is a professor)
5. Enrollment.StudentID -> Student.StudentID
(each enrollment record belongs to one student)
6. Enrollment.CourseID -> Course.CourseID

(each enrollment record refers to one course)

Part 2: ER Diagram Construction

Task 2.1: Hospital Management System



1. Entities:

Strong entities: Patient, Doctor, Department

Weak entities: Room (depends on Department), Appointment (depends on Patient), Prescription (depends on Doctor)

2. Attributes by type:

Simple: Name, Birthdate, OfficeLocation, DeptName, Location, RoomNumber (partial key), Dosage, Instructions, Purpose, Notes, AppointmentDate, AppointmentTime

Composite: Address (Street, City, State, Zip)

Multivalued: Phone (Patient), Specialization (Doctor)

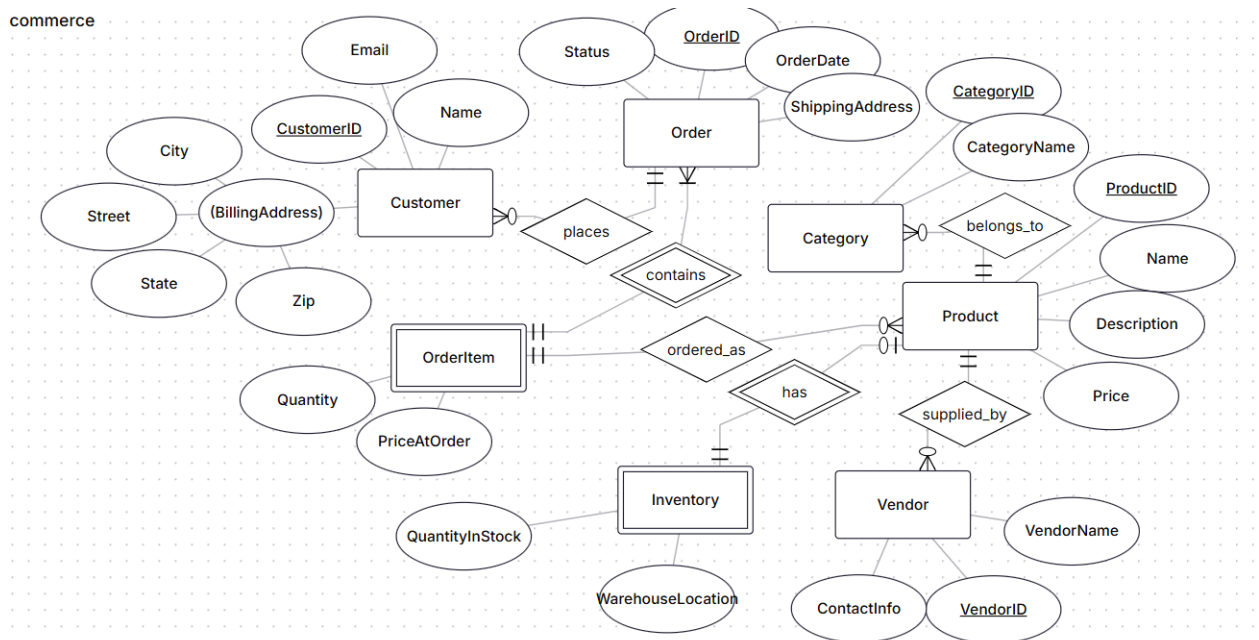
Key: PatientID, DoctorID, DeptCode

3. Relationships and cardinalities:

1. Doctor works_in Department — N:1
2. Room located_in Department — N:1 (identifying)
3. Patient has Appointment — 1:N (identifying)
4. Doctor conducts Appointment — 1:N
5. Doctor writes Prescription — 1:N (identifying)
6. Patient receives Prescription — 1:N

Task 2.2: E-commerce Platform

commerce



1. Create a complete ER diagram

2. Weak entity and justification:

OrderItem is a weak entity. It cannot exist independently. Its primary key is composite: OrderID + ProductID.

3. Many-to-many relationship requiring attributes:

Order and Product have a many-to-many relationship: one order can contain many products, and one product can appear in many orders.

Part 4: Normalization Workshop

Task 4.1: Denormalized Table Analysis

StudentProject(StudentID, StudentName, StudentMajor, ProjectID, ProjectTitle, ProjectType, SupervisorID, SupervisorName, SupervisorDept, Role, HoursWorked, StartDate, EndDate)

1. Identify functional dependencies:

StudentID → StudentName, StudentMajor

ProjectID → ProjectTitle, ProjectType, SupervisorID

SupervisorID → SupervisorName, SupervisorDept

StudentID, ProjectID → Role, HoursWorked, StartDate, EndDate

2. Identify problems: - What redundancy exists in this table? - Give specific examples of update, insert, and delete anomalies:

Redundancy: StudentName and StudentMajor repeat in every row for the same student's different projects. SupervisorName and SupervisorDept repeat in every row where that supervisor appears.

Update: If a supervisor changes department, every row containing that SupervisorID must be updated.

Insert: A new project cannot be recorded until at least one student is assigned to it — there is no row to store ProjectTitle or SupervisorID otherwise.

Delete: If the last student working on a project is removed from the table, all information about that project is lost entirely.

3. Apply 1NF:

No 1NF violations exist — every cell holds a single atomic value, and there are no repeating groups. The table is already in 1NF.

4. Apply 2NF:

Primary Key: StudentID, ProjectID

Partial dependencies:

StudentID -> StudentName, StudentMajor

ProjectID -> ProjectTitle, ProjectType, SupervisorID, SupervisorName, SupervisorDept

2NF decomposition:

student(StudentID, StudentName, StudentMajor)

project(ProjectID, ProjectTitle, ProjectType, SupervisorID, SupervisorName, SupervisorDept)

student_project(StudentID, ProjectID, Role, HoursWorked, StartDate, EndDate)

5. Apply 3NF:

Transitive dependency:

In the project table: ProjectID -> SupervisorID -> SupervisorName, SupervisorDept. Since SupervisorID determines SupervisorName and SupervisorDept, and SupervisorID is itself a non-key attribute in project, this is a transitive dependency.

Final 3NF Decomposition:

student(StudentID, StudentName, StudentMajor)

supervisor(SupervisorID, SupervisorName, SupervisorDept)

project(ProjectID, ProjectTitle, ProjectType, SupervisorID)

student_project(StudentID, ProjectID, Role, HoursWorked, StartDate, EndDate)

Task 4.2: Advanced Normalization

Given Table:

CourseSchedule(StudentID, StudentMajor, CourseID, CourseName, InstructorID, InstructorName, TimeSlot, Room, Building)

Business rules:

- Each student has exactly one major
- Each course has a fixed name
- Each instructor has exactly one name
- Each time slot in a room determines the building (rooms are unique across campus)
- Each course section is taught by one instructor at one time in one room
- A student can be enrolled in multiple course sections

1. Primary Key:

Primary Key: StudentID, CourseID, TimeSlot

A course section is uniquely defined by CourseID + TimeSlot. Each course section is taught by one instructor at one time in one room. Since a student can enroll in multiple sections, StudentID must also be part of the key to identify which student attends which section.

2. Functional Dependencies:

StudentID -> StudentMajor

CourseID -> CourseName

InstructorID -> InstructorName

Room, TimeSlot -> Building

CourseID, TimeSlot -> InstructorID, Room

3. BCNF Check:

Functional Dependency	Left side a superkey?	Verdict
StudentID -> StudentMajor	No	Violates BCNF
CourseID -> CourseName	No	Violates BCNF
InstructorID -> InstructorName	No	Violates BCNF
Room, TimeSlot -> Building	No	Violates BCNF
CourseID, TimeSlot -> InstructorID, Room	No	Violates BCNF

the table is not in BCNF and not even in 2NF, since StudentMajor and CourseName depend on only part of the primary key.

4. BCNF Decomposition:

student(StudentID, StudentMajor)

course(CourseID, CourseName)

instructor(InstructorID, InstructorName)

room_building(Room, TimeSlot, Building)

section(CourseID, TimeSlot, InstructorID, Room)

enrollment(StudentID, CourseID, TimeSlot)

5. Potential Loss of Information:

The decomposition does not lose any information. If we join all six tables back together, we get exactly the original table again.

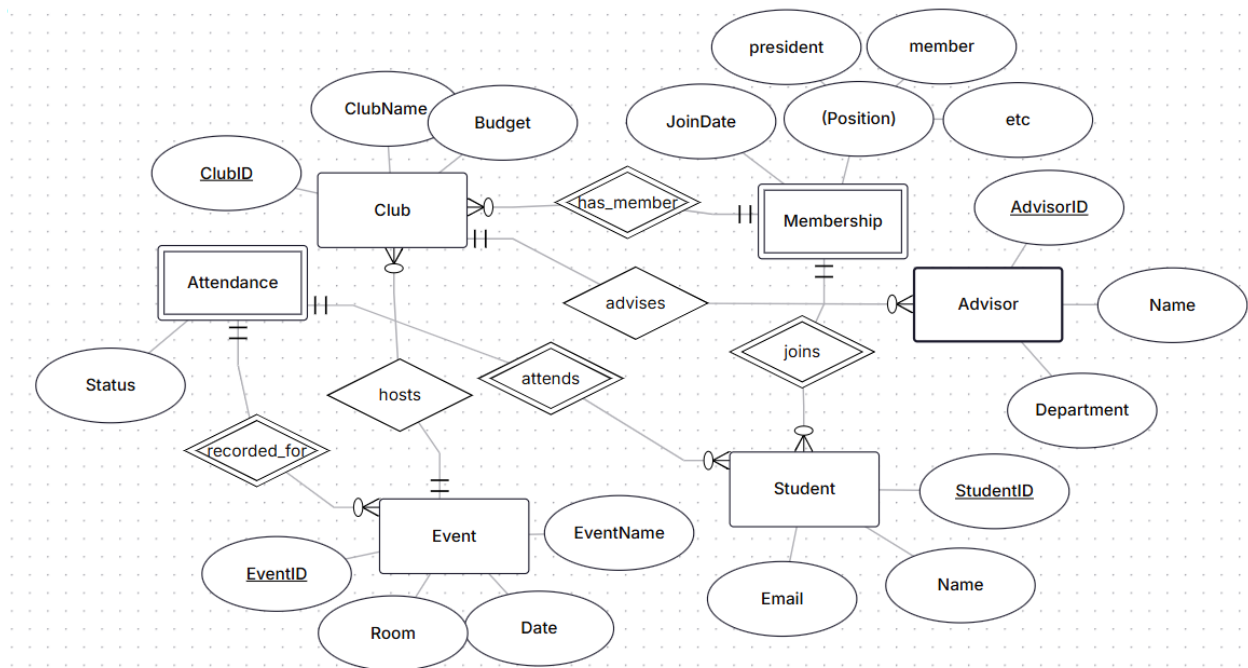
Part 5: Design Challenge

Task 5.1: Real-World Application

System Requirements:

- Student clubs and organizations information
- Club membership (students can join multiple clubs, clubs have multiple members)
- Club events and student attendance tracking
- Club officer positions (president, treasurer, secretary, etc.)
- Faculty advisors for clubs (each club has one advisor, faculty can advise multiple clubs)
- Room reservations for club events
- Club budget and expense tracking

1. Create a complete ER diagram for this system:



2. Convert your ER diagram to a normalized relational schema:

student(StudentID, Name, Email)

club(ClubID, ClubName, Budget, AdvisorID)

advisor(AdvisorID, Name, Department)

event(EventID, EventName, Date, Room, ClubID)

membership(StudentID, ClubID, Position, JoinDate)

attendance(StudentID, EventID, Status)

3. Identify at least one design decision where you had multiple valid options and explain your choice:

Officer positions could be a separate entity, or just an attribute inside membership. I chose the attribute approach, since a position is just a property of one student's membership — it doesn't need its own table.

4. Write 3 example queries that your database should support (in English, not SQL):

1. Find all students who are club presidents.

2. Show all upcoming events and their rooms.
3. Count how many members each club has.